

Синхронный и асинхронный вызов метода БЛ в облаке

Для работы с синхронными и асинхронными вызовами методов БЛ применяются классы Object ([Python](#) / [C++](#)) и EndPoint ([Python](#) / [C++](#)).

При асинхронном вызове [метода БЛ](#) программа не дожидается выполнения метода, а лишь запрашивает его результат, обработку которого производит другая функция. Применение асинхронных вызовов позволяет рационально распределить использование ресурсов системы в тех случаях, когда результат выполнения метода не требуется для дальнейшего функционирования программы, например, фиксация вызова метода в логах сервиса.

В зависимости от сервиса облака, к которому происходит обращение для вызова метода бизнес-логики, выделяют два [типа вызовов](#): **локальные** и **удалённые**.

Вызов любых методов осуществляется посредством протоколов **AMQP**, **RBC** или **HTTP/HTTPS**. Какой протокол будет использован для вызова зависит от используемого [типа доставки](#).

Для обработки результата асинхронного вызова может быть установлен [обработчик](#).

Список модулей для работы с асинхронными вызовами

Подключите сервисный модуль BL Async в состав тех сервисов (на вызываемой и вызывающей сторонах), где планируется использование механизма асинхронных вызовов.

Конфигурация, необходимая для работы модуля, будет получена автоматически из облака вместе с остальной конфигурацией для работы сервиса.

Для работы с асинхронными вызовами добавьте в конфигурацию зависимости от следующих модулей:

- AMQP Client — реализация работы с "брокером" асинхронных вызовов RabbitMQ. **Если используется протокол AMQP.**
- RequestBrokerClient - реализация работы с платформенным "брокером" асинхронных вызовов [request-broker](#). **Если используется протокол RBC.**
- BL Async — основной модуль по работе с асинхронными вызовами.
- Berkeley DB — библиотека для работы с Berkeley DB.
- Berkeley Storage — локальное временное хранилище асинхронных вызовов на основе Berkeley DB.
- MessageBroker — базовый модуль для работы с произвольным "брокером".

Типы вызовов метода

Локальный вызов

При локальном вызове выполнение метода БЛ производится в том же сервисе облака, из которого он был вызван. Объект класса EndPoint ([Python](#) / [C++](#)) не используется, либо передан как пустая строка.

- C++

```
1 bl::Object object( L"НазваниеОбъекта" );
2
3 // Синхронный вызов.
4 // Вызов метода происходит в текущем процессе и потоке.
5 int result = object.Invoke< int >( L"НазваниеМетода", param1, param2
6 );
7
8 // Асинхронный вызов.
9 // Вызов метода происходит в каком-либо процессе того же сервиса
10 (возможно, на другом сервере).
```

```
9 object.AsyncInvoke( L"НазваниеМетода", param1, param2 );
```

- Python

```
1 obj = BLObject('НазваниеОбъекта')
2
3 # Синхронный вызов.
4 # Вызов метода происходит в текущем процессе и потоке.
5 # альтернативный способ вызова: result =
  НазваниеОбъекта.НазваниеМетода(param1, param2).
6 result = obj.Invoke('НазваниеМетода', param1, param2)
7
8 # Асинхронный вызов.
9 # Вызов метода происходит в каком-либо процессе того же сервиса
  (возможно, на другом сервере).
10 obj.AsyncInvoke('НазваниеМетода', param1, param2)
```

Удалённый вызов

При удалённом вызове метод БЛ выполняется в другом сервисе облака. Объект класса EndPoint используется в обязательном порядке.

Для работы с удалёнными вызовами установлены требования, описанные в статье [Необходимые условия для удалённого вызова метода бизнес-логики](#).

Для вызова можно задать заголовок запроса, опции вызываемого метода, приоритет вызова, проверку сессии, число повторных вызовов и их таймаут. Подробнее об этом читайте в описании класса EndPoint ([Python](#) / [C++](#)).

- C++

```
1 blcore::EndPoint ep{ L"НазваниеСервисаПриложения" };
2 blcore::Object obj{ L"НазваниеОбъекта", ep };
3
4 // Синхронный вызов
5 int result = obj.Invoke< int >( L"НазваниеМетода", param1, param2 );
6
7 // Асинхронный вызов
8 obj.AsyncInvoke( L"НазваниеМетода", param1, param2 );
```

- Python

```
1 ep = EndPoint('НазваниеСервисаПриложения')
2 obj = BLObject('НазваниеОбъекта', ep)
3
4 # Синхронный вызов
5 result = obj.Invoke('НазваниеМетода', param1, param2)
6
7 # Асинхронный вызов
8 obj.AsyncInvoke('НазваниеМетода', param1, param2)
9
1 # Метод запрашивает список сертификатов по предоставленной
  пользователем лицензии
2 def GetCertsByLicense(client_id, license):
3     # Создаём параметры фильтрации
4     filter = Record()
5     filter.AddInt32('Аккаунт', client_id)
6     filter.AddInt32('Лицензия', license)
7
8     # Создаём пустые объекты навигации и сортировки
9     nav = Navigation()
10    sort = SortingList()
11
12    # Вызываем метод в зависимости от наличия адреса в конфигурации
```

```

13     if ConfigGet('Клиент инсайд.Адрес') == '':
14         return Сертификат.СписокПоЛицензии(None, filter, sort, nav)
15     else:
16         # Синхронный вызов
17         return BLObject('Сертификат', 'Клиент
инсайд').Invoke('СписокПоЛицензии', None, filter, sort, nav)

```

EndPoint позволяет выполнить вызов не только по системному имени сервиса, но и по абсолютному URL. Однако в этом случае могут возникнуть проблемы при аутентификации, т.к. политики AuthByUserID, AuthByClientID и т.п. работать не будут.

Пример абсолютного URL: <https://test-online.sbis.ru/service/>

Аутентификация при удалённом вызове

В объект [EndPoint](#) можно явно передать сессию последним параметром sid. Если сессия не установлена, то при внешнем вызове будет использоваться текущая сессия (если таковая имеется).

Вызов метода по виртуальному каталогу сервиса с предварительной аутентификацией

Если вызов производится по виртуальному каталогу сервиса, и из сервиса отсутствует возможность вызвать метод без предварительной аутентификации, то:

- Она будет произведена под служебным пользователем при условии, что сервис не расположен в корне приложения и не является сервисом персональных данных клиентов.
- Если сервис "персональный", то при каждом вызове требуется дополнительно устанавливать пользователя или клиента, под которым должен выполняться вызов. Это необходимо, чтобы получить доступ к данным клиента. Если сервис входит одновременно в сайты online/inside/private, то вызов будет производиться через online-private.sbis.ru (сайт для межсервисных обращений на сервисы с системным именем "online").

Конфигурацию пользователя/клиента производят в объекте Endpoint, примеры использования которого вы можете прочитать в [этом разделе](#). В объекте передают второй параметр auth_data:

- AuthByClientID — по прямому идентификатору клиента в облаке;
- AuthByExtID — по внешнему идентификатору клиента из биллинга;
- AuthByUserID — по прямому идентификатору пользователя клиента в облаке;
- AuthByLogin — по логину пользователя клиента в облаке.

Примеры использования

Вызов с использованием системного имени / виртуального каталога сервиса.

- C++

```

1 // Используем системное имя/виртуальный каталог сервиса
2 bl::Object obj( L"Phonebook", L"telephony" );
3 int result = obj.Invoke< int >( L"ReadNumber" , param1, param2 );

```

- Python

```

1 # Вариант вызова в старом стиле
2 # Используем системное имя/виртуальный каталог сервиса
3 result = BLObject( 'Phonebook', 'telephony' ).Invoke( 'ReadNumber',
param1, param2)

1 # Вариант вызова в новом стиле
2 # Используем системное имя/виртуальный каталог сервиса
3 result = EndPoint( 'telephony' ).Phonebook.ReadNumber( param1, param2
)

```

Если вы обращаетесь к "Основной сервис" (его системное имя "online") по параметру "ОсновнойСервис", то исходный код можно изменить следующим образом: создаете endpoint — объект для идентификации сервиса-получателя вызова, указывая в параметре "service_name" системное имя сервиса, а в параметре "auth_data" — способ получения клиента.

- C++

```

1 #include <sbis-lib/sbis-cloud-
interaction/auth/user_id_auth_policy.hpp>

```

```

2
3 // Аутентификация по прямому идентификатору клиента в облаке
4 bl::EndPoint ep( L"online", AuthByUserID(myId) );
5 String response = bl::Object( L"MyBLOName", ep ).Invoke< String >(
  L"MyMethodName", param1, param2 );

```

- Python

```

1 # Старый вариант использования
2 ep = EndPoint( service_name='online', auth_data=AuthByUserID(myId) )
3 response = BLObject( 'MyBLOName', ep ).Invoke( 'MyMethodName',
  param1, param2 )

1 # Новый вариант использования
2 ep = EndPoint( service_name='online', auth_data=AuthByUserID(myId) )
3 response = ep.MyBLOName.MyMethodName( param1, param2 )

```

Приоритет обработки вызова метода БЛ

Вызовы методов БЛ обрабатываются Брокером, который осуществляет обмен сообщениями между компонентами программной системы. Для веб-приложений СБИС в качестве брокера применяется [RabbitMQ](#), либо платформенный [request-broker](#), который обрабатывает вызовы в соответствии с их приоритетами.

Для **синхронных вызовов** приоритет обработки можно задать с помощью методов [sbis.EndPoint.SetPriority\(\)](#) (при работе в Python) или [sbis::blcore::EndPoint::SetPriority\(\)](#) (при работе в C++).

Приоритеты делятся на следующие типы:

- **grNORMAL** (по умолчанию) — нормальный приоритет.

Вызов отправляется на основной пул.

Используйте такой приоритет для "быстрых" методов БЛ, результат выполнения которых требуется моментально возвращать пользователю.

- **grLOW** — пониженный приоритет. Вызов отправляется на служебный пул.

Используйте такой приоритет для фоновых задач или "долгих" методов БЛ (например, при работе с файлами).

У всех **асинхронных вызовов** приоритет **grLOW** и изменить его нельзя, т.е. все вызовы падают только на служебный пул. Также для асинхронных вызовов можно задать [приоритет доставки брокера](#).

Таймаут вызова метода БЛ

Подробно описано в статье [Таймаут для методов бизнес-логики](#).

Порядок асинхронного вызова методов

Не гарантируется, что асинхронный вызов "Метод1" будет выполнен раньше, чем "Метод2".

```

1 // Пример на C++
2 bl::Object obj( L"Объект" );
3
4 // Асинхронно вызываем два метода бизнес-логики
5 obj.AsyncInvoke( L"Метод1" );
6 obj.AsyncInvoke( L"Метод2" );

1 # Пример на Python
2 obj = sbis.BLObject( "Объект" )
3
4 # Асинхронно вызываем два метода бизнес-логики
5 obj.AsyncInvoke( "Метод1" )
6 obj.AsyncInvoke( "Метод2" )

```

Чтобы явно задать последовательность вызова, применяют цепочку вызовов посредством callback (см. [Обработка результатов асинхронного вызова](#)). Поэтому предыдущий примером нужно преобразовать следующим образом:

1. Вызываем метод с обработчиком:

```
1 // Пример на C++
2 bl::Object obj( L"Объект" );
3 obj.AsyncInvoke( L"Метод1", bl::Callbacks(
  L"Объект.ОбработчикНаУспех", nullptr ) );

1 # Пример на Python
2 obj = sbis.BLObject( "Объект" )
3 obj.AsyncInvoke( "Метод1", callbacks = [ "Объект.ОбработчикНаУспех" ]
  )
```

2. Метод-обработчик "Объект.УспешнаяРегистрацияВхода" в свою очередь будет выполнять последующий вызов:

```
1 // Пример на C++
2 obj.AsyncInvoke( L"Метод2" );

1 # Пример на Python
2 obj.AsyncInvoke( "Метод2" )
```

Типы доставки асинхронного вызова методов

Существует два способа асинхронного вызова методов, которые конфигурируются вторым аргументом объекта класса EndPoint ([Python](#) / [C++](#)).

Вызов с негарантированной доставкой

Вызов с негарантированной доставкой будет обработан не более одного раза. Это означает, что он может быть утерян, например, если сервис-исполнитель будет остановлен до исполнения запроса.

Запрос отправляется по протоколу **HTTP/HTTPS**. Адрес и другие реквизиты удаленного сервиса диспетчер берет из конфигурации. Соответственно, если мы отправляем запрос на сервис "Администрирование", то в конфигурации должна быть секция "Администрирование", в которой должен быть указан URL данного сервиса, и, опционально, реквизиты для аутентификации на нем. При отсутствии этой секции в настройках будет генерироваться исключение.

Вызов осуществляется посредством существующих возможностей библиотеки sbis-communication прямо из рабочего процесса БЛ (sbis-rpc-worker) с минимальным таймаутом. Управление клиентскому коду возвращается сразу после получения запроса исполнителем, ещё до передачи его в обработку.

Вызов с гарантированной доставкой

Применяется по умолчанию для асинхронных вызовов.

Запрос отправляется по протоколу **AMQP** или **RBC**. Вызов с гарантированной доставкой будет обработан не менее одного раза. Это означает, что он может быть повторен, например, если сервис-исполнитель во время исполнения будет остановлен. Число повторных вызовов задаётся через [SetRepeatCnt](#).

При использовании гарантированной доставки вызов осуществляется через Брокер.

Обработка поступивших запросов ведется в общем порядке приоритетов, которые задаются при помощи метода [SetPriority](#).

Вызов производится непосредственно из рабочего процесса БЛ (sbis-rpc-worker). Управление клиентскому коду возвращается в тот момент, когда от Брокера будет получено подтверждение приема сообщения.

Выбор протокола

Существует два протокола отправки:

- AMQP. Запрос будет доставлен посредством RabbitMQ. Данный протокол используется по умолчанию.
- RBC. Запрос доставляется через платформенный [брокер асинхронного обмена](#).

Пример вызова:

- C++

```
1 auto ep = EndPoint('ОсновнойСервис', True, session)
2
```

```

3 # Задаем RBC протокол, AMQP используется по умолчанию
4 ep.SetTransportProtocol( rbc::RBC_PROTOCOL,
   rpc::InteractionKind::AsyncInvokeWithGuarantee )
5
6 obj = BLObject('ЧастноеЛицо', ep)
7 obj.AsyncInvoke('ЗаписатьПерсональныеДанные', transData)

```

- Python

```

1 ep = EndPoint('ОсновнойСервис', True, session)
2
3 # Задаем RBC протокол, AMQP используется по умолчанию
4 ep.SetTransportProtocol('Async Request Broker',
   sbis.InteractionKind.AsyncInvokeWithGuarantee)
5
6 obj = BLObject('ЧастноеЛицо', ep)
7 obj.AsyncInvoke('ЗаписатьПерсональныеДанные', transData)

```

Специализированный брокер

Для определенного класса запросов можно выделить отдельный изолированный контур доставки. Специализированный выделенный брокер будет обслуживать только такие запросы, что повысит отказоустойчивость. В случае аварий на основном брокере, данный контур продолжит работать, и наоборот. Это необходимо прикладным механизмам, для которых крайне критична потеря запросов, либо продуцирующим большой поток запросов, чтобы уменьшить влияние на остальных.



Данная фича доступна только при работе по протоколу RBC.

Для создания Специализированного брокера обратитесь к [ответственным](#).

Повышенные гарантии доставки

Запросу, проходящему через специализированный брокер также можно повысить гарантию доставки. Все запросы будут храниться в резервном хранилище и будут повторяться, если по какой то причине были потеряны самим брокером (например, во время аварий). Опция необходима для особо критичных запросов, потеря которых крайне нежелательна.

Пример вызова:

- C++

```

1 auto ep = EndPoint('ОсновнойСервис', True, session)
2
3 # Задаем настройки с выделенным брокером, RBC протокол установится
   автоматически
4 auto rbc_settings = std::make_unique<
   rbc::RbcTransportProtocolSettings >();
5 # Задаем специализированный брокер
6 rbc_settings->SetBrokerName( L"custom_broker_name" );
7 # Задаем повышенную гарантию, если требуется
8 ep.SetHighDeliveryGuarantee( true );
9 ep.SetTransportProtocol( std::move( rbc_settings ),
   rpc::InteractionKind::AsyncInvokeWithGuarantee );
10
11 obj = BLObject('ЧастноеЛицо', ep)
12 obj.AsyncInvoke('ЗаписатьПерсональныеДанные', transData)

```

- Python

```

1 ep = EndPoint('ОсновнойСервис', True, session)
2
3 # Задаем настройки с выделенным брокером, RBC протокол установится
   автоматически
4 rbc_settings = sbisrequestbrokerclient.RbcTransportProtocolSettings()

```

```

5 # Задаем специализированный брокер
6 rbc_settings.BrokerName = 'custom_broker_name'
7 # Задаем повышенную гарантию, если требуется
8 rbc_settings.HighDeliveryGuarantee = True
9 ep.SetTransportProtocol(rbc_settings,
10 sbis.InteractionKind.AsyncInvokeWithGuarantee)
11 obj = BLObject('ЧастноеЛицо', ep)
12 obj.AsyncInvoke('ЗаписатьПерсональныеДанные', transData)

```

Подписка на специализированный брокер

Подписка на основные брокеры происходит по умолчанию. Однако, если используется специализированный брокер, то принимающему и вызывающему сервису необходимо самостоятельно подписаться на него. Делать это необходимо в обработчике события загрузки модуля.

- C++

```

1 #include <sbis-request-broker-
  client/rbc_transport_protocol_settings.hpp>
2
3 struct SomeInitializer : Id::ModuleInitializerHandle
4 {
5     bool OnEndLoadModules( EndLoadEventAnswer& /*answer*/ ) const
6     override
7     {
8         rbc::AddBrokerToListen( L"custom_broker_name"_sv );
9         return true;
10    }

```

- Python

```

1 import sbisrequestbrokerclient
2
3 def OnEndAllLoadModules(answer):
4     """Обработчик события загрузки модуля триггеров"""
5
6     sbisrequestbrokerclient.AddBrokerToListen('custom_broker_name')

```

Приоритет доставки асинхронного вызова через брокер

Асинхронные вызовы доставляются до сервисов бизнес-логики через брокер, в котором образуется очередь. По умолчанию, все новые вызовы встают в конце очереди. Для каждого асинхронного вызова можно задать расширенные настройки приоритета в общей очереди с помощью метода [SetAsyncPriority](#). Метод принимает целое число от 0 (наименьший приоритет) до 9 (наивысший приоритет). По умолчанию значение 2.



Не путайте SetAsyncPriority с SetPriority!

SetPriority выбирает пул (основной или служебный), для асинхронных вызовов пул всегда служебный. Если вызван метод SetPriority и указан приоритет NORMAL, то для SetAsyncPriority значение приоритета будет 5.

При асинхронном запросе пул (независимо от [типа доставки](#)) всегда служебный.

Пример вызова:

```

1 # Асинхронный вызов метода с гарантированной доставкой и заданным
  числом повторений.
2 # Устанавливаем вызов с гарантированной доставкой и сессию, под которой
  нужно вызвать метод.
3 ep = EndPoint('ОсновнойСервис', True, session)
4

```



```

5 # Задаём число повторений вызова в случае ошибочного вызова.
6 ep.SetRepeatCnt(10)
7
8 # Устанавливаем приоритет доставки вызова через Брокер.
9 ep.SetAsyncPriority(8)
10
11 # Значение приоритета определено ответственным за ресурс.
12 obj = BLObject('ЧастноеЛицо', ep)
13 obj.AsyncInvoke('ЗаписатьПерсональныеДанные', transData)

```

Ограничения на размер тела запроса



Размер тела запроса не должен превышать 100 Кбайт! Когда размер тела запроса находится в диапазоне от 100 Кбайт и до 1 Мбайт, в логи записывается сообщение с типом **Предупреждение**.

Когда размер тела запроса превышает порог в **1 Мбайт**, в логи записывается сообщение с типом **Ошибка** и текстом вида:

"Асинхронный вызов метода "ОбработкаPDF.ОбработкаФайлов" с сервиса "private.sbis.ru/pdfservice/service/" имеет слишком большой размер тела запроса (4677805 байт). Идентификатор запроса: 8c78ef0c-9a91-4935-b3d0-69fa5f42d961".

По этой причине запрещена передача файлов асинхронным вызовом с гарантией доставки.

Логи доступны в реестре **Журнал логов** → [Отчет по ошибкам](#) сервиса "Управление облаком":

Облако

с 28.09.23 12:23:14 по 28.09.23 12:28:54, С средним уровнем порога 80% x

Приложения

Домовы

Кластеры

Серверы

Анализ работы

Журнал логов

Отчет по ошибкам

Очередь

Анализ входов

Настройки

Клиенты

Пользователи

Описание (байт)	Метод	Описание (байт)	Статус	Описание (байт)	Клиент
51 / 10	InvokeRecursiveProcessor.Process1 (Пользователь Б.В.)	92 / 16	400	for-for portal (Пользователь Б.В.)	78 / 15 Компания "Текстор", ООО
33 / 6	ATPCClientProcessor.ProcessEvent (Пользователь М.А.)	31 / 6	400	FindContractor (Бизнесменский Б.С.)	192 / 26 Неизвестный клиент
20 / 4	FSInteractionProcessor.ReceiveAnswerProcess2 (Пользователь Б.В.)	7 / 1	400	RequestContract (Пользователь Б.В.)	1 / 0 МСП Банк, АО
18 / 4	FSInteractionProcessor.SendProcess2 (Пользователь Б.В.)	4 / 1	400	CalculateContract (Бизнесменский Б.С.)	1 / 0 Платформа Нормы, ООО
7 / 3	ReceivingDocFromFSProcessor.Process1 (Пользователь М.А.)	513 / 106	400	evb_u117 (Пользователь Б.В.)	1 / 0 Наликс, ООО
6 / 3	Person.GetMNBYPassport1 (Бизнесменский А.П.)	183 / 37	400	evb_u120 (Пользователь Б.В.)	1 / 0 ООО "СМЕДИКАЛ"
4 / 3	FindPurchasesProcessor.Process (Пользователь М.А.)	187 / 27	400	evb_u124 (Пользователь Д.Б.)	1 / 0 ООО "СОСВЕЧАТЬ"
1 / 0	FindContractor.GetData1 (Бизнесменский Б.С.)	125 / 25	400	evb_u130 (Пользователь Д.Б.)	1 / 0 г. Томск, МКУ ЦС МТОУ
1 530 / 306	RequirementsConverter.OkRequirementsConversion (Пользователь Б.С.)	113 / 23	400	evb_u136 (Пользователь Б.В.)	1 / 0 Ютек, ООО
129 / 26	Неизвестный метод	502 / 20	400	evb_u159 (Пользователь Д.Б.)	414 / 83 Регулюма Вера Владимировна
		99 / 20	400	evb_u160 (Пользователь Д.Б.)	104 / 33 Алексон, ООО
		90 / 16	400	evb_u122 (Пользователь Д.Б.)	130 / 26 Неизвестный клиент
		53 / 11	400	evb_u132 (Пользователь Д.Б.)	93 / 19 Компания "Сравент", ООО
		46 / 9	400	evb_u121 (Пользователь Д.Б.)	92 / 16 Энергосервис Эксперт, ООО
		6 / 1	400	personal-data (Бизнесменский А.П.)	79 / 16 Уральские электрические сети, ООО
		1 / 0	400	edc-its-contractors (Пользователь Б.В.)	78 / 16 Матгедор Елена, ООО
		41 / 8	400	Logi (Бизнесменский Б.С.)	71 / 14 СКА-МАСТЕР
		40 / 8	400	Service presentation (Бизнесменский А.С.)	63 / 13 РЦ Культуры, МУП
		19 / 4	400	business-calendar	54 / 11 СтанцияРесурс-регион, ООО

Причины ограничения:

1. Брокер реализован как мульти-мастер: каждый запрос сохраняется на диск на каждой из нод, что для файлов "дорого".
2. Большие запросы увеличивают время синхронизации между нодами, что увеличивает время ответа Брокера.
3. Брокер не рассчитан на хранение данных и у него выделен небольшой диск под хранилище. Если в асинхронных запросах передавать много данных, а целевой сервис не будет успевать обрабатывать запросы, то хранилище переполнится, что приведет к нестабильности всего облака.

Для передачи больших данных используйте протокол [Huge Payload Protocol](#).

Приоритет обработчиков результата асинхронного вызова

В большинстве случаев, если результат выполнения метода ожидает пользователь (например, когда он загружает файлы), приоритет должен быть нормальным (значение 5), а если никто не ожидает, то — пониженный (значение 2).

"Расширенные" значения приоритетов асинхронных вызовов требуются для отдельных специализированных сервисов, где их определяет ответственный за сервис.

При асинхронном вызове внутри вызванного метода иногда требуется узнать приоритет, с которым он был вызван. Это нужно например для того, чтобы последующие вызовы из этого метода сделать с таким же приоритетом. Приоритет

можно получить через вызов:

```
Session.GetHeader( "X-AsyncPriority" )
```

Для синхронного запроса вернется None, для асинхронного — строка с приоритетом.

Если при вызове были указаны обработчики асинхронного вызова, то они будут выполнены с тем приоритетом, что и сам вызов.

Передача больших данных в асинхронных вызовах. Huge Payload Protocol

Для передачи больших данных в асинхронных вызовах используйте протокол Huge Payload Protocol, в котором реализован следующий подход:

- Если размер данных запроса не превышает пороговое значение (по умолчанию 100 Кбайт), то вызов происходит по обычной схеме (см. [гарантированная](#) и [негарантированная](#) доставка).
- Если превышено пороговое значение, то:
 - на вызывающей стороне: данные выгружаются на [Сервис временных файлов](#) (file-transfer), а в запросе передаётся только ссылка на данные.
 - на вызываемой стороне: перед вызовом исходного кода метода БЛ данные загружаются из сервиса file-transfer и передаются в метод так, как будто они были получены напрямую.

Протокол оформлен в виде сервисных модулей HugePayload и HugePayload-Py, которые поставляются в составе [SBIS SDK](#).

Модуль HugePayload-Py применяется, когда вызов производится из исходного кода Python. Включите модули в состав сервиса, из которого вы создаётся асинхронный вызов метода БЛ, а также в сервис, в котором находится вызываемый метод БЛ.

Чтобы асинхронный вызов происходил с использованием Huge Payload Protocol, явно установите использование протокола с помощью метода SetProtocol():

- Python

```
1  # Подключаем библиотеку для работы с протоколом Huge Payload
   Protocol.
2  import hugepayload
3
4  # Задаём сервис, из которого будет выполняться вызов метода БЛ.
5  e = EndPoint('service_name')
6
7  # Задаём использование протокола Huge Payload Protocol.
8  e.SetProtocol(hugepayload.GetHugePayloadRpcProtocol())
9
10 # Выполняем асинхронный вызов метода БЛ, в аргументы которого
    передаём большие данные.
11 BLObject('object', e).AsyncInvoke('method', big_data)
```

- C++

```
1  // Подключаем заголовочные файлы для работы с протоколом Huge Payload
   Protocol.
2  #include <sbis-huge-payload-protocol/protocol_traits.hpp>
3  #include <sbis-bl-core/object.hpp>
4  # Задаём сервис, из которого будет выполняться вызов метода БЛ.
5  blcore::EndPoint e( L"service_name" );
6
7  // Задаём использование протокола Huge Payload Protocol.
8  e.SetProtocol<rpc::HugePayloadRpcProtocol>();
9
10 // Выполняем асинхронный вызов метода БЛ, в аргументы которого
```

```
11 передаём большие данные.  
blcore::Object( L"object", e ).AsyncInvoke( L"method", big_data );
```

Также в CMakeLists.txt вашей библиотеки необходимо добавить зависимость:

```
sbis_add_sdk_dependencies(sbis-huge-payload-protocol)
```

Обработка результатов асинхронного вызова

Во многих сценариях требуется обработать результат асинхронного вызова метода БЛ. Всего существует четыре сценария:

Сценарий	Поведение
Падение/завершение диспетчера рабочих процессов при работе без гарантии вызова	Результат никак не обрабатывается. Является штатным сценарием при асинхронном вызове с негарантированной доставкой.
Падение рабочего процесса, в котором выполнялся метод	Является аварийным сценарием и представляет опасность для сервера приложений. В случае критической ошибки при выполнении задачи (падение рабочего процесса с дампом) делается соответствующая запись в лог, а задача помещается в специальную очередь ошибок Брокера для группы серверов (называется она как основная очередь группы с добавлением постфикса #error). Повторная обработка сбойной задачи не предусмотрена (после исправления ошибки разработчик, ответственный за данную предметную область, должен будет переместить необработанные задачи обратно в основную очередь).
Необработанное (дошедшее до кода платформы) исключение при работе метода	Для обработки результата используется специальная функция — обработчик асинхронного вызова, который создают на обработку ошибки.
Нормальное завершение работы, возврат результата	Для обработки результата используется специальная функция — обработчик асинхронного вызова, который создают на обработку результат нормального выполнения метода.

Для C++ обработчики могут быть заданы любым параметром при помощи [bl::Callbacks](#). В нём первый аргумент — это обработчик результата успешного выполнения метода (callback), а второй аргумент — обработчик ошибки (errback). Если один из обработчиков задавать не требуется, то вместо его названия добавляем nullptr.

```
1 bl::Object obj( L"НазваниеОбъекта" );  
2 obj.AsyncInvoke( L "НазваниеМетода", param, bl::Callbacks(  
  L"Объект.ОбработчикУспеха", L"Объект.ОбработчикОшибки" ) );
```

Для Python обработчик может быть задан только именованным параметром callbacks. В нём первый аргумент — это обработчик результата успешного выполнения метода (callback), а второй аргумент — обработчик ошибки (errback). Если один из обработчиков задавать не требуется, то вместо его названия добавляем None или пустую строку.

```
1 obj = BLObject( 'НазваниеОбъекта' )  
2 obj.AsyncInvoke( 'НазваниеМетода', param, callbacks =  
  ( 'Объект.ОбработчикУспеха', 'Объект.ОбработчикОшибки' ) )
```

Однако зачастую в callbacks нужно понять, к какой высокоуровневой операции данный callback или errback относится. Обычно данная информация не выводится из параметров результата вызываемого метода. Соответственно, необходимо передать некоторые пользовательские данные в составе callbacks для того, чтобы понять какая операция была выполнена.

Для решения этой проблемы в callbacks передается третий параметр - КонтекстОперации. Это строка, в которую можно закодировать необходимые данные для восстановления информации о выполненной операции.

Примеры:

```

1  # без errback-a
2  obj = BLObject('НазваниеОбъекта')
3  obj.AsyncInvoke('НазваниеМетода', param, callbacks =
4    ('Объект.ОбработчикУспеха', '', '{"report_id":42, "attachment_id":12}'))
5  # без callback-a
6  obj = BLObject('НазваниеОбъекта')
7  obj.AsyncInvoke('НазваниеМетода', param, callbacks = ('',
8    'Объект.ОбработчикОшибки', '{"report_id":42, "attachment_id":12}'))
9  # callback без контекста операции
10 obj.AsyncInvoke('НазваниеМетода', param, callbacks =
11   ('Объект.ОбработчикУспеха'))
12 # errback без контекста операции
13 obj.AsyncInvoke('НазваниеМетода', param, callbacks =
14   ('', 'Объект.ОбработчикОшибки', ''))
15 # errback с контекстом операции
16 obj.AsyncInvoke('НазваниеМетода', param, callbacks =
17   ('', 'Объект.ОбработчикОшибки', '{"report_id":42, "attachment_id":12}'))

```

В составе [параметра Метод](#) эта строка придет в поле context, если она была задана.

Обработчик асинхронного вызова



Об обработчиках асинхронного вызова

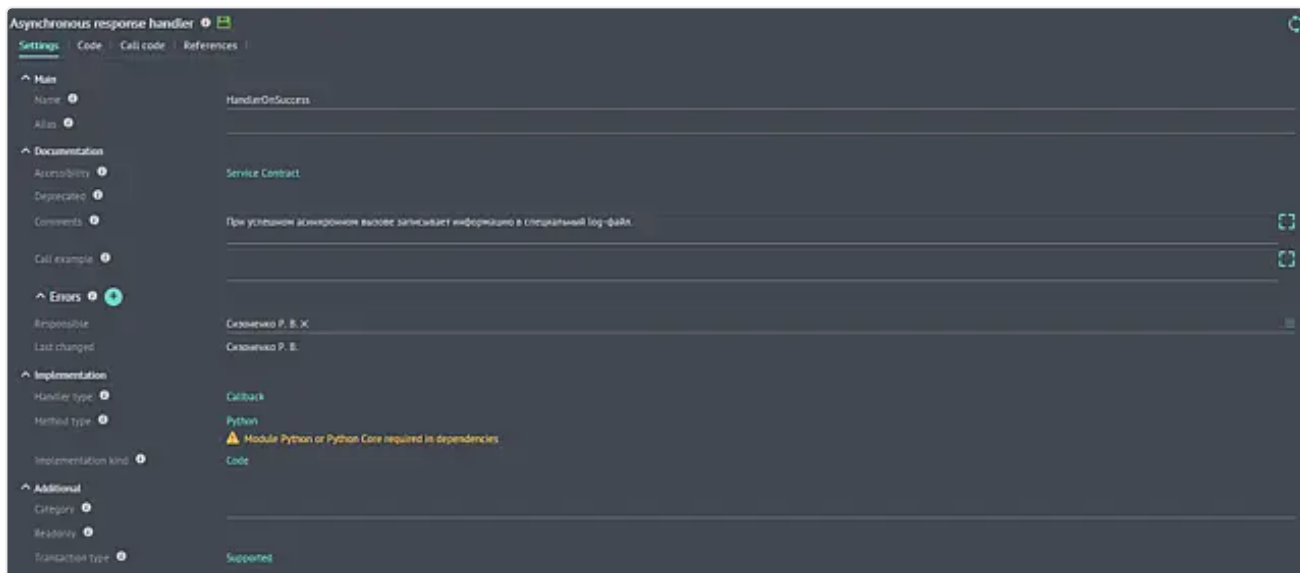
Платформой СБИС гарантируется только то, что указанные функции-обработчики будут вызваны ровно один раз. Никакой обработки результата вызова данных функций не предусмотрено.

Обработчик асинхронного вызова — это специальная функция, предназначенная для обработки либо успешного выполнения асинхронного запроса, либо его ошибки. Использовать обработчики следует в том случае, если требуется выполнить некоторую дополнительную работу на сервисе-инициаторе запроса по завершении выполнения данного запроса исполнителем.

Обработчик может быть создан для:

1. Обработки результата успешного асинхронного запроса (Callback). Функции-обработчики результата нормального выполнения метода принимают на вход три параметра: "Метод", "Параметры" и "Результат". Callback вызывается асинхронно.
2. Обработки ошибочного выполнения асинхронного запроса (Errback). Функции-обработчики ошибок принимают на вход три параметра: "Метод", "Параметры" и "Ошибка".

Создание обработчиков производят через [Genie](#) в [справочниках объектов](#) сервиса-инициатора. Интерфейс создания обработчика выглядит следующим образом:



Параметры функций-обработчиков

Параметр	Тип	Описание
1. Метод	sbis.Record/sbis::Record	Текстовые поля записи: - НазваниеМетода – название метода, для которого был вызван обработчик. - Сессия – сессия, под который был исполнен метод. - Сервис – асинхронная очередь сервиса, исполнившего метод. - ГруппаСерверов – название группы серверов, исполнившей метод. - request_uuid – идентификатор запроса. Совпадает с идентификатором, который был возвращен при вызове AsyncInvoke. - context – строка с контекстом операции, переданным вызывающей стороной при задании callback-a/errback-a
2. Параметры	sbis.Record/sbis::Record	Обращаться к параметрам лучше по номерам, а не именам, т.к. в некоторых случаях параметры могут быть безымянными (см. таблицу Обработка результата выполнения запроса на исполнителе). <pre> 1 # Пример обращения к параметрам по имени или порядковому номеру. 2 >>> from sbis_root import * 3 >>> r = </pre>

		<pre> Record({"name": "value"}) 4 >>> r.name 5 'value' 6 >>> str(r['name']) 7 'value' 8 >>> str(r[0]) 9 'value' </pre>
3. Результат	sbis.Record/sbis::Record	Единственное поле с именем "Результат" произвольного типа.
4. Ошибка	sbis.Record/sbis::Record	Содержит поля, описывающие произошедшее в методе исключение: • Ошибка – текст ошибки. То, что в JSON приходит в details. • Сообщение – текст пользовательского сообщения. То, что в JSON приходит в message. • ДопИнфо – дополнительная информация об исключении. Требуется для корректного восстановления экземпляра исключения. • Ид – идентификатор сериализуемого исключения. • Type – тип исключения "error" или "warning". • ErrorCode – код исключения.

В обработке ошибки можно повторить сериализуемое исключение, которое привело к ошибке. Для этого нужно вызвать [GenerateAsyncException](#).

Примеры

Обработчик ошибочного выполнения запроса, реализованный на Python. Он используется для записи события в логи планировщика задач:

```

1 @staticmethod
2 def MyHandler( Метод, Параметры, Ошибка ):
3     SqlQuery(
4         """update scheduler_log set reply_date = now(),
5         reply_stage = 2,
6         reply_status = $1 where task_uuid = $2::uuid""",
7         'Выполнение завершилось с ошибкой: ' + Ошибка.Ошибка,
8         Метод.request_uuid
9     )

```

Асинхронные очереди Брокера

Под каждый сервис бизнес-логики на Брокере создается несколько очередей. Их названия формируются сервисом при старте.

Основная очередь

Название основной очереди имеет вид <UUID_Облака>.<ИдГруппыСерверов>. Все символы в нижнем регистре.

- UUID_Облака хранится в базе управления облаком в таблице **ServiceGuid** и присылается на сервис вместе с конфигурацией в параметре Ядро.Асинхронные сообщения.ИдентификаторОблака;
- **ИдГруппыСерверов** – идентификатор группы серверов из структуры облака в шестнадцатеричном виде. Присылается на сервис вместе с конфигурацией в параметре __sbis__group_id.

Название основной очереди сохраняется в параметре Ядро.Асинхронные сообщения.НазваниеОчереди и имеет следующий вид:

```
Ядро.Асинхронные сообщения.НазваниеОчереди=4704ea8e-22b9-40e0-96c9-c18a4ec6ceb2.00000008
```

Для **локальных стендов** (серверов, не входящих в структуру облака) название основной очереди имеет вид <КороткоеДоменноеИмяКомпьютера>:<Порт><Каталог>, например:

```
tsd-abramovvi:1002/admin/service/
```

Прочие очереди

Кроме основной очереди также создаются:

- очередь ошибок – <ИмяОсновнойОчереди>#error
- отложенные очереди – <ИмяОсновнойОчереди>#delay<ВремяПовтора>

Обработка результата выполнения запроса на исполнителя

Результат	Перенос запроса в отложенную очередь	Перенос запроса в очередь ошибок	Вызов Callback'a	Вызов Errback'a	Уменьшение числа повторов	Примечание
На методе задана аутентификация, но сервис аутентификации временно не доступен. Либо невозможно проверить сессию из-за временно не доступного сервиса администрирования.	+					
На методе заданы ошибочные данные для аутентификации, либо не удастся восстановить сессию (пользователь удален или заблокирован).		+		+		Параметры в errback'e могут быть безымянными.
При		+		+		Параметры в

обработке запроса «упал» рабочий процесс.						errmsg'e могут быть безымянными.
На методе задано число повторов и при обработке запроса произошла ошибка.	+				+	
При обработке запроса произошла ошибка.				+		Параметры в errmsg'e будут названы как в репозитории.
Не удалось вызвать метод из-за несоответстви я сигнатуры.				+		Параметры в errmsg'e будут безымянными.
Метод отработал до конца, код возврата 200.			+			